

LẬP TRÌNH WEB (WEBPR330479)

Restful API trong Java Servlet

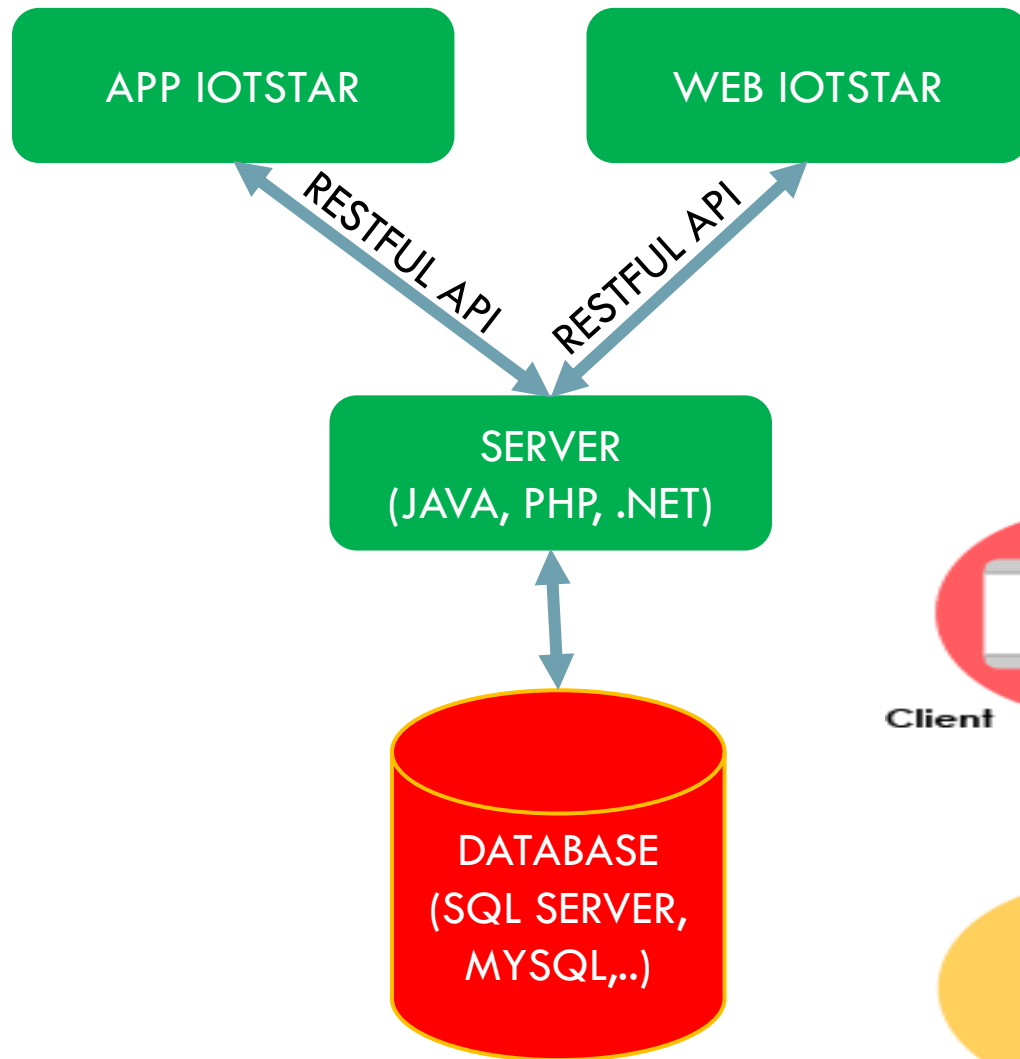


THS. NGUYỄN HỮU TRUNG

- Ths. Nguyễn Hữu Trung
- Khoa Công Nghệ Thông Tin
- Trường Đại học Sư Phạm Kỹ Thuật TP.HCM
- 090.861.7108
- trungnh@hcmute.edu.vn
- <https://www.youtube.com/@baigiai>



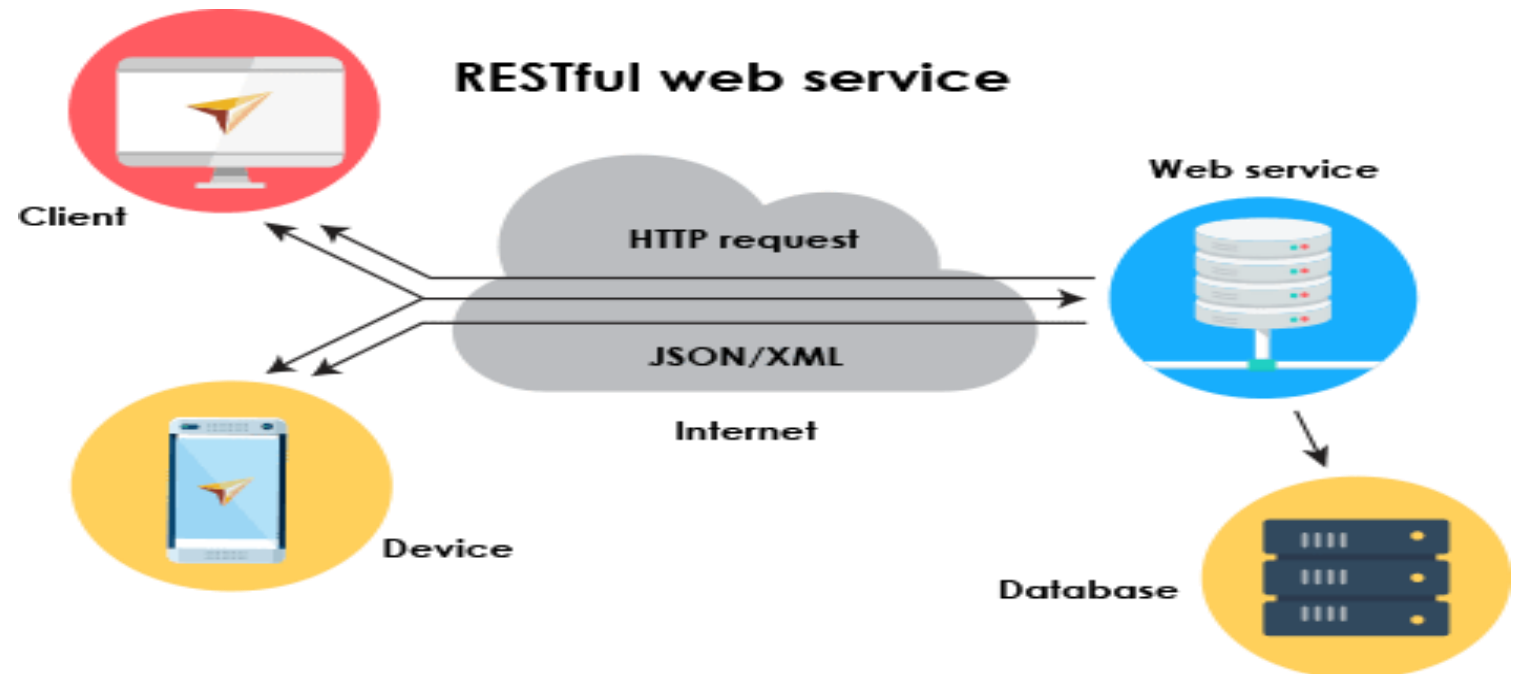
- Restful API
- Jackson và ví dụ
- Gson và ví dụ
- Restful API trong Spring boot
- Render Rest API to AJAX



Web Service là tập hợp các giao thức và tiêu chuẩn mở được sử dụng để trao đổi dữ liệu giữa các ứng dụng hoặc giữa các hệ thống.

Web service thì có 2 dạng:

- SOAP: Sử dụng XML
- REST: Sử dụng JSon



- API (Application Programming Interface): là một giao diện lập trình phần mềm trung gian hỗ trợ các ứng dụng giao tiếp với nhau.
- Chúng cho phép các ứng dụng khác giao tiếp hiệu quả với nhau và dùng được cho web-based system, operating system, database system, computer hardware, or software library.
- Web API là mô hình được tạo ra nhằm mục đích hỗ trợ MVC như: routing, controller, action result, filter, IoC container, model binder, unit test, injection. Đồng thời còn cung cấp tính năng cho phép Restful đầy đủ các phương thức: **Get/ Post/ Put/ Delete** dữ liệu.

- **RESTful API** là một tiêu chuẩn dùng trong việc thiết kế API cho các ứng dụng web (thiết kế Web services) để tiện cho việc quản lý các resource. Nó chú trọng vào tài nguyên hệ thống (tệp văn bản, ảnh, âm thanh, video, hoặc dữ liệu động...), bao gồm các trạng thái tài nguyên được định dạng và được truyền tải qua HTTP.
- Dưới đây là 4 lệnh phổ biến nhất cho phép người dùng truy cập vào RESTful API:
 - ▣ **GET**: cho phép truy vấn object
 - ▣ **POST**: giúp tạo object mới
 - ▣ **PUT**: hỗ trợ sửa đổi hoặc thay thế một object
 - ▣ **DELETE**: loại bỏ một object
- Hiện tại có 3 cơ chế Authorize chính:
 - ▣ HTTP Basic
 - ▣ JSON Web Token (JWT)
 - ▣ OAuth2

- Khi chúng ta request một API nào đó thường thì sẽ có vài status code để nhận biết sau:
 - ▣ 200 OK – Trả về thành công cho những phương thức GET, PUT, PATCH hoặc DELETE.
 - ▣ 201 Created – Trả về khi một Resource vừa được tạo thành công.
 - ▣ 204 No Content – Trả về khi Resource xóa thành công.
 - ▣ 304 Not Modified – Client có thể sử dụng dữ liệu cache.
 - ▣ 400 Bad Request – Request không hợp lệ
 - ▣ 401 Unauthorized – Request cần có auth.
 - ▣ 403 Forbidden – bị từ chối không cho phép.
 - ▣ 404 Not Found – Không tìm thấy resource từ URI
 - ▣ 405 Method Not Allowed – Phương thức không cho phép với user hiện tại.
 - ▣ 410 Gone – Resource không còn tồn tại, Version cũ đã không còn hỗ trợ.
 - ▣ 415 Unsupported Media Type – Không hỗ trợ kiểu Resource này.
 - ▣ 422 Unprocessable Entity – Dữ liệu không được xác thực
 - ▣ 429 Too Many Requests – Request bị từ chối do bị giới hạn

- **JSON**(JavaScript **O**bject **N**otation) là một kiểu định dạng dữ liệu tuân theo một quy luật nhất định mà hầu hết các ngôn ngữ lập trình hiện nay đều có thể đọc được. **JSON** là một tiêu chuẩn mở để trao đổi dữ liệu trên web.

XML

```
<empinfo>
  <employees>
    <employee>
      <name>James Kirk</name>
      <age>40</age>
    </employee>
    <employee>
      <name>Jean-Luc Picard</name>
      <age>45</age>
    </employee>
    <employee>
      <name>Wesley Crusher</name>
      <age>27</age>
    </employee>
  </employees>
</empinfo>
```

JSON

```
{ "empinfo" :
  {
    "employees" : [
      {
        "name" : "James Kirk",
        "age" : 40,
      },
      {
        "name" : "Jean-Luc Picard",
        "age" : 45,
      },
      {
        "name" : "Wesley Crusher",
        "age" : 27,
      }
    ]
  }
}
```

- JSON sử dụng các cặp *key – value* để định dạng. Nó hỗ trợ các cấu trúc dữ liệu như đối tượng và mảng.

Ví dụ một tập tin có tên `iotstar.json` với nội dung như ở dưới đây sử dụng format kiểu JSON để lưu trữ thông tin:

```
{ "name" : "iotstar",  
  "title" : "Nguyễn Hữu Trung"  
}
```

- Hai thư viện Json phổ biến: Gson và Jackson (Spring boot đã được tích hợp)



- Jackson là một thư viện Java chứa rất nhiều chức năng để đọc và xây dựng JSON. Nó có khả năng ràng buộc dữ liệu rất mạnh mẽ và cung cấp một framework để tùy chỉnh quá trình chuyển đổi tượng Java sang chuỗi JSON và chuỗi JSON sang đối tượng Java.

```
<!-- https://mvnrepository.com/artifact/com.fasterxml.jackson.core/jackson-core -->
```

```
<dependency>
```

```
  <groupId>com.fasterxml.jackson.core</groupId>
```

```
  <artifactId>jackson-databind</artifactId>
```

```
  <version>2.13.0</version>
```

```
</dependency>
```

□ Phân tích chuỗi JSON

▣ Có 3 cách thường dùng để phân tích chuỗi JSON sang Java:

- **Streaming** : sử dụng JsonParser để phân tích cú pháp json. Nó cung cấp các phần tử json như là các token. Sử dụng JsonGenerator để tạo ra json từ chuỗi, số nguyên, boolean, ...
- **Tree Traversing** : json có thể được đọc thành một JsonNode. Node sau đó có thể đi qua để có được thuộc tính cần thiết. Một cây cũng có thể được tạo ra và sau đó được viết như chuỗi json.
- **Data Binding** (liên kết dữ liệu) : sử dụng Annotation để đánh dấu các thuộc tính trên đối tượng liên kết với các phần tử của chuỗi JSON.

❑ Chuyển Java Object sang JSON

```
ObjectMapper mapper = new ObjectMapper();
```

```
// Convert object to JSON string and save into a file directly
```

```
mapper.writeValue(new File("data/result.json"), category); // Plain JSON
```

```
// Convert object to JSON string
```

```
String jsonInString = mapper.writeValueAsString(category);
```

```
System.out.println("JSON: " + jsonInString);
```

```
// Convert object to JSON string and pretty print
```

```
jsonInString = mapper.writerWithDefaultPrettyPrinter().writeValueAsString(category);
```

```
System.out.println("JSON pretty print: " + jsonInString);
```

□ Phân tích JSON sang Java Object

```
ObjectMapper mapper = new ObjectMapper();
```

```
// Convert JSON string from file to Object
```

```
CategoryModel category = mapper.readValue(new File("data/result.json"), CategoryModel.class);
```

```
System.out.println(category);
```

```
// Convert JSON string to Object
```

```
String jsonInString = "{\n  \"id\":1,\n  \"name\": \"Quần áo nam\"}";
```

```
student = mapper.readValue(jsonInString, CategoryModel.class);
```

```
System.out.println(category);
```

```
// Pretty print
```

```
jsonInString = mapper.writerWithDefaultPrettyPrinter().writeValueAsString(category);
```

```
System.out.println(jsonInString);
```

□ Chuyển Json Array sang Java List

```
ObjectMapper mapper = new ObjectMapper();
```

```
CategoryModel category1 = getCategory(1);
```

```
CategoryModel category2 = getCategory(2);
```

```
CategoryModel[] categorys = { category1, category2 };
```

```
// Convert array object to JSON string
```

```
String jsonInString = mapper.writerWithDefaultPrettyPrinter().writeValueAsString(categorys);
```

```
System.out.println("JSON: " + jsonInString);
```

```
// Convert a JSON Array format to a Java List object
```

```
List<CategoryModel> list = mapper.readValue(jsonInString, new TypeReference<List<CategoryModel>>() {  
});
```

```
for (CategoryModel category : list) {  
    System.out.println(category);  
}
```

❑ Chuyển Json Object sang Java Map

```
ObjectMapper mapper = new ObjectMapper();
```

```
CategoryModel cate = new CategoryModel(21, "Hữu Trung");
```

```
// Convert array object to JSON string
```

```
String jsonInString = mapper.writerWithDefaultPrettyPrinter().writeValueAsString(cate);
```

```
System.out.println("JSON: " + jsonInString);
```

```
// Convert a JSON to a Map
```

```
Map<String, Object> map = mapper.readValue(jsonInString, new TypeReference<Map<String, Object>>() {  
});
```

```
System.out.println("Map: " + map);
```

- Bước 1: cấu hình thư viện Jackson trong file pom.xml

```
<!-- json -->
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-databind</artifactId>
  <version>2.13.0</version>
</dependency>
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-core</artifactId>
  <version>2.13.0</version>
</dependency>
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-annotations</artifactId>
  <version>2.13.0</version>
</dependency>
```

```
<dependency>
  <groupId>org.codehaus.jackson</groupId>
  <artifactId>jackson-mapper-asl</artifactId>
  <version>1.9.13</version>
</dependency>
<dependency>
  <groupId>org.codehaus.jackson</groupId>
  <artifactId>jackson-core-asl</artifactId>
  <version>1.9.13</version>
</dependency>
<!-- end json -->
```

□ Bước 2: Tạo Model Category

```
public class CategoryModel implements Serializable {  
    private int id;  
    private String name;  
    private String icon;  
  
    //tạo constructor và Getter/Setter  
  
}
```

□ Bước 3: Tạo hàm thư viện

```
package vn.iotstar.util;
import java.io.BufferedReader;
import java.io.IOException;
import com.fasterxml.jackson.databind.ObjectMapper;
public class HttpUtil {
    private String value;
    public HttpUtil (String value) {
        this.value = value;
    }
    public <T> T toModel(Class<T> tClass) {
        try {
            return new ObjectMapper().readValue(value, tClass);
        } catch (Exception e) {
            System.out.print(e.getMessage());
        }
        return null;
    }
}
```

```
public <T> List<T> listModel(Class<T> tClass) {
    List<T> list = new ArrayList<T>();
    try {
        list.add(new ObjectMapper().readValue(value,
            tClass));
    } catch (Exception e) {
        System.out.print(e.getMessage());
    }
    return list;
}
```

□ Bước 3: Tạo hàm thư viện

```
public static HttpUtil of (BufferedReader reader) {  
    StringBuilder sb = new StringBuilder();  
    try {  
        String line;  
        while ((line = reader.readLine()) != null) {  
            sb.append(line);  
        }  
    } catch (IOException e) {  
        System.out.print(e.getMessage());  
    }  
    return new HttpUtil(sb.toString());  
}
```

- Bước 4: Tạo các hàm Insert, getAll, Edit, Delete trong Service

Các bài trước đã viết

- Bước 5: Tạo Controller

```
package vn.iotstar.api.admin;
import java.io.IOException;
import java.util.List;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import com.fasterxml.jackson.databind.ObjectMapper;
import vn.iotstar.model.CategoryModel;
import vn.iotstar.service.CategoryService;
import vn.iotstar.service.impl.CategoryServiceImpl;
import vn.iotstar.util.HttpUtil;
```

```
@WebServlet(urlPatterns = {"/api-
admin-category"})
```

```
public class CategoryApiController
```

```
extends HttpServlet{
```

```
    CategoryService categoryService =
```

```
    new CategoryServiceImpl();
```

```
    private static final long
```

```
    serialVersionUID =
```

```
    7733176626714241908L;
```

□ Bước 5: Tạo Controller

```
@Override
protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws ServletException,
IOException {
    ObjectMapper mapper = new ObjectMapper();
    req.setCharacterEncoding("UTF-8");
    resp.setContentType("application/json");
    List<CategoryModel> cateModel = HttpUtil.of(req.getReader()).listModel(CategoryModel.class);
    cateModel = categoryService.getAllCategory();
    mapper.writeValue(resp.getOutputStream(), cateModel);
}
```

□ Bước 5: Tạo Controller

@Override

```
protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws ServletException,
IOException {
    ObjectMapper mapper = new ObjectMapper();
    req.setCharacterEncoding("UTF-8");
    resp.setContentType("application/json");
    CategoryModel cateModel = HttpUtil.of(req.getReader()).toModel(CategoryModel.class);
    cateModel = categoryService.insert(cateModel);
    mapper.writeValue(resp.getOutputStream(), "{Đã thêm thành công}");
}
```

□ Bước 5: Tạo Controller

@Override

```
protected void doPut(HttpServletRequest req, HttpServletResponse resp) throws ServletException,  
IOException {
```

```
    ObjectMapper mapper = new ObjectMapper();
```

```
    req.setCharacterEncoding("UTF-8");
```

```
    resp.setContentType("application/json");
```

```
    CategoryModel cateModel = HttpUtil.of(req.getReader()).toModel(CategoryModel.class);
```

```
    cateModel = categoryService.edit(cateModel);
```

```
    mapper.writeValue(resp.getOutputStream(), "{Đã cập nhật thành công}");
```

```
}
```

□ Bước 5: Tạo Controller

@Override

```
protected void doDelete(HttpServletRequest req, HttpServletResponse resp) throws ServletException,
IOException {
    ObjectMapper mapper = new ObjectMapper();
    req.setCharacterEncoding("UTF-8");
    resp.setContentType("application/json");
    CategoryModel cateModel = HttpUtil.of(req.getReader()).toModel(CategoryModel.class);
    categoryService.delete(cateModel.getId());
    mapper.writeValue(resp.getOutputStream(), "{Đã xóa}");
}
```

□ Bước 6: Test kết quả bằng Postman:doGet

The screenshot shows the Postman interface with a GET request to `http://localhost:8090/BanHang/api-admin-category`. The response status is 200 OK, with a time of 1144 ms and a size of 1.31 KB. The response body is displayed in JSON format, showing an array of 4 category objects.

Query Params

KEY	VALUE	DESCRIPTION
Key	Value	Description

Body | Cookies | Headers (5) | Test Results

Status: 200 OK | Time: 1144 ms | Size: 1.31 KB | [Save Response](#)

Pretty | Raw | Preview | Visualize | JSON |  

```
1  [
2    {
3      "id": 1,
4      "name": "Quần Áo Nam",
5      "icon": "category/687f3967b7c2fe6a134a2c11894eea4b_tn.png"
6    },
7    {
8      "id": 2,
9      "name": "Quần Áo Nữ",
10     "icon": "category/687f3967b7c2fe6a134a2c11894eea4b_tn.png"
11   },
12   {
13     "id": 3,
14     "name": "Quần Áo Trẻ Em",
15     "icon": "category/687f3967b7c2fe6a134a2c11894eea4b_tn.png"
16   },
17   {
18     "id": 4,
```

□ Bước 6: Test kết quả bằng Postman: doPost

The screenshot displays the Postman interface for a POST request. The URL bar shows `http://localhost:8090/BanHang/api-admin-category`. The 'Body' tab is selected, showing a JSON payload:

```
{  "name": "Quần Áo Nam",  "icon": "category/687f3967b7c2fe6a134a2c11894eea4b_tn.png"}
```

. The 'Send' button is visible in the top right. Below the request, the 'Response' tab is active, showing a status of `200 OK` with a time of `57 ms` and a size of `183 B`. The response body is `"{Đã thêm thành công}"`.

POST `http://localhost:8090/BanHang/api-admin-category` Send

Params Authorization Headers (8) **Body** Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded **raw** binary GraphQL **JSON** Beautify

```
1 {
2   .... "name": "Quần Áo Nam",
3   .... "icon": "category/687f3967b7c2fe6a134a2c11894eea4b_tn.png"
4 }
```

Body **Cookies** Headers (5) Test Results Status: 200 OK Time: 57 ms Size: 183 B Save Response

Pretty Raw Preview Visualize **JSON**

```
1 "{Đã thêm thành công}"
```

□ Bước 6: Test kết quả bằng Postman: doPut

The screenshot shows the Postman interface for a PUT request. The URL is `http://localhost:8090/BanHang/api-admin-category`. The request body is a JSON object:

```
1 {  
2   ... "id": 28,  
3   ... "name": "Quần Áo Nam Nam",  
4   ... "icon": "category/687f3967b7c2fe6a134a2c11894eea4b_tn.png"  
5 }
```

The response status is 200 OK, with a time of 134 ms and a size of 190 B. The response body is:

```
1 "{Đã cập nhật thành công}"
```

❏ Bước 6: Test kết quả bằng Postman: doDelete

The screenshot shows the Postman interface for a DELETE request. The URL is `http://localhost:8090/BanHang/api-admin-category`. The request body is a JSON object: `{ "id": 23 }`. The response status is 200 OK, with a time of 25 ms and a size of 169 B. The response body is `{ "Đã xóa" }`.

Request Tab:

- Method: DELETE
- URL: `http://localhost:8090/BanHang/api-admin-category`
- Body Type: JSON
- Body:

```
1 {  
2   "id": 23  
3 }
```

Response Tab:

- Status: 200 OK
- Time: 25 ms
- Size: 169 B
- Body:

```
1 "{Đã xóa}"
```



Gson là một thư viện Java có thể được sử dụng để **chuyển đổi các đối tượng Java thành chuỗi JSON**. Nó cũng có thể được sử dụng để **chuyển đổi một chuỗi JSON thành một đối tượng Java** tương ứng. Gson có thể làm việc với các đối tượng Java tùy ý bao gồm các đối tượng đã tồn tại từ trước mà bạn không có mã nguồn của nó.

- Cung cấp các phương thức **toJson()** và **fromJson()** đơn giản để chuyển các đối tượng Java sang JSON và ngược lại.
- Cho phép các **đối tượng không thể thay đổi** có sẵn được chuyển đổi sang/ từ JSON.
- Hỗ trợ rộng rãi của Java **Generics**.
- Cho phép **tùy chỉnh** cho các đối tượng để chuyển đổi sang/ từ JSON.
- Hỗ trợ các **đối tượng phức tạp** (có phân cấp thừa kế nhiều cấp và sử dụng rộng rãi các kiểu dữ liệu Generics).

```
<!-- https://mvnrepository.com/artifact/com.google.code.gson/gson -->  
<dependency>  
  <groupId>com.google.code.gson</groupId>  
  <artifactId>gson</artifactId>  
  <version>2.8.6</version>  
</dependency>
```

```
import com.google.gson.Gson;
public class PrimitiveExample {
    public static void main(String[] args) {
        // Serialization
        Gson gson = new Gson();
        gson.toJson(40); // ==> 40
        gson.toJson("Trung"); // ==> "Trung"
        gson.toJson(new Long(38)); // ==> 38
        int[] values = { 39 };
        gson.toJson(values); // ==> [39]
        // Deserialization
        int one = gson.fromJson("40", int.class); // ==> 40
        Integer oneInteger = gson.fromJson("38", Integer.class); // ==> 38
        Long oneLong = gson.fromJson("39", Long.class); // ==> 39
        Boolean bool = gson.fromJson("false", Boolean.class); // ==> false
        String str = gson.fromJson("\"Trung\"", String.class); // ==> "Trung"
        String[] anotherStr = gson.fromJson("[\"trung\"]", String[].class); // ==> ["Trung"]
    }
}
```

```
public class CategoryModel implements Serializable {  
    private int id;  
    private String name;  
    private String icon;  
}
```

```
public static void main(String[] args) {  
    CategoryModel cate = new CategoryModel(1, "Quần áo nam");  
    Gson gson = new Gson();  
    // Serialization  
    String json = gson.toJson(cate);  
    System.out.println(json); // {"id":1,"name":"Quần áo nam"}  
  
    // Deserialization  
    CategoryModel cate1 = gson.fromJson(json, CategoryModel.class);  
    System.out.println(cate1);  
}
```

```
public static void main(String[] args) {  
    Gson gson = new Gson();  
    int[] int1 = { 10, 20, 30, 40, 50 };  
    String[] string1 = { "Trung", "Hữu", "Nguyễn" };  
  
    // Serialization  
    gson.toJson(int1); // ==> [10,20,30,40,50]  
    gson.toJson(string1); // ==> ["Trung", "Hữu", "Nguyễn"]  
  
    // Deserialization  
    int[] int2 = gson.fromJson("[10,20,30,40,50]", int[].class);  
    // ==> int2 will be same as int1  
}
```

```
public static void main(String[] args) {  
    Gson gson = new Gson();  
    List<Integer> int1 = Arrays.asList(10,20,30,40,50);  
  
    // Serialization  
    String json = gson.toJson(int1); // ==> json is [10,20,30,40,50]  
  
    // Deserialization  
    Type collectionType = new TypeToken<List<Integer>>().getType();  
    Collection<Integer> int2 = gson.fromJson(json, collectionType);  
    // ==> int2 is same as int1  
}
```

```
public static void main(String[] args) {  
    // Gson gson = new Gson();  
    Gson gson = new GsonBuilder().setPrettyPrinting().create();  
    List<CategoryModel> list = new ArrayList<CategoryModel>();  
    list.add(new CategoryModel(1, "Quần áo nam"));  
    list.add(new CategoryModel(2, "Quần áo nữ"));  
    // Serialization  
    String json = gson.toJson(list);  
    System.out.println(json);  
  
    // Deserialization  
    Type collectionType = new TypeToken<List<CategoryModel>>().getType();  
    List<CategoryModel> list2 = gson.fromJson(json, collectionType);  
    // ==> list2 is same as list  
}
```

GsonBuilder().setPrettyPrinting().create() để tạo chuỗi
Json có format.

```
public static void main(String[] args) {  
    Map<String, String> inMap = new HashMap<String, String>();  
    inMap.put("name", "Trung");  
    inMap.put("site", "http://iotstar.vn");  
    System.out.println("Input Map: " + inMap);  
    Gson gson = new Gson();  
    // Serialization  
    String json = gson.toJson(inMap);  
    System.out.println("Json: " + json); // {"site":"http://iotstar.vn","name":"Trung"}  
  
    // Deserialization  
    Type mapType = new TypeToken<Map<String, Object>>(){}.getType();  
    Map<String, String> outMap = gson.fromJson(json, mapType);  
    System.out.println("Output Map: " + outMap);  
}
```

- Bước 1: Cấu hình thư viện Gson trong file pom.xml

```
<!-- Gson -->
<dependency>
  <groupId>com.google.code.gson</groupId>
  <artifactId>gson</artifactId>
  <version>2.8.6</version>
</dependency>
<!-- End Gson -->
```

- Bước 2: Tạo Model CategoryModel

```
public class CategoryModel implements Serializable {
  private int id;
  private String name;
  private String icon;

  //tạo Constructor, getter/setter

}
```

□ Bước 3: Viết hàm doGet trong Controller

@Override

protected void doGet(HttpServletRequest req, HttpServletResponse resp) **throws**

ServletException, IOException {

//định dạng chuỗi dạng json

resp.setContentType("application/json");

resp.setCharacterEncoding("UTF-8");

List<CategoryModel> list = **new** ArrayList<CategoryModel>();

//gọi hàm lấy toàn bộ category trong Service

list = categoryService.getAllCategory();

//Khởi tạo đối tượng Gson

Gson gson = **new** Gson();

//Gọi hàm toJson để chuyển Object thành String Json

String cateJson = gson.toJson(list);

//hiển thị kết quả

PrintWriter out = resp.getWriter();

out.write(cateJson);

out.close();

}

□ Bước 3: Viết hàm doPost trong Controller

```
@Override
protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws ServletException,
IOException {
    resp.setContentType("application/json");
    resp.setCharacterEncoding("UTF-8");
    //String json = '{"name':'Trung','icon':'category/a2c11894eea4b tn.png}';
    Gson gson = new Gson(); //Khởi tạo đối tượng Gson
    //gọi hàm fromJson để chuyển từ String sang Object
    CategoryModel cateJson = gson.fromJson(req.getReader(), CategoryModel.class);
    //gọi hàm insert trong Service để thêm dữ liệu đối tượng vào database
    cateJson= categoryService.insert(cateJson);
    //Viết thông báo kết quả
    PrintWriter out = resp.getWriter();
    out.println("Đã thêm thành công");
    out.close();
}
```

□ Bước 3: Viết hàm doPut trong Controller

@Override

protected void doPut(HttpServletRequest req, HttpServletResponse resp) **throws** ServletException, IOException {

```
    resp.setContentType("application/json");
```

```
    resp.setCharacterEncoding("UTF-8");
```

```
    //String json = "{ 'name': 'Trung', 'icon': 'category/687f3967b94eea4b tn.png' }";
```

```
    Gson gson = new Gson(); // Khởi tạo đối tượng Gson
```

```
    // gọi hàm fromJson để chuyển từ String sang Object
```

```
    CategoryModel cateJson = gson.fromJson(req.getReader(), CategoryModel.class);
```

```
    // gọi hàm edit trong Service để cập nhật dữ liệu đối tượng vào database
```

```
    cateJson = categoryService.edit(cateJson);
```

```
    //Viết thông báo kết quả
```

```
    PrintWriter out = resp.getWriter();
```

```
    out.println("Đã sửa thành công");
```

```
    out.close();
```

```
}
```

❑ Bước 3: Viết hàm doDelete trong Controller

@Override

protected void doDelete(HttpServletRequest req, HttpServletResponse resp) **throws** ServletException, IOException {

```
    resp.setContentType("application/json");
```

```
    resp.setCharacterEncoding("UTF-8");
```

```
    //String json = '{"name':'Trung','icon':'category/687f894eea4b tn.png'}';
```

```
    Gson gson = new Gson(); //Khởi tạo đối tượng Gson
```

```
    //gọi hàm fromJson để chuyển từ String sang Object
```

```
    CategoryModel cateJson = gson.fromJson(req.getReader(), CategoryModel.class);
```

```
    //gọi hàm insert trong Service để thêm dữ liệu đối tượng vào database
```

```
    categoryService.delete(cateJson.getId());
```

```
    //Viết thông báo kết quả
```

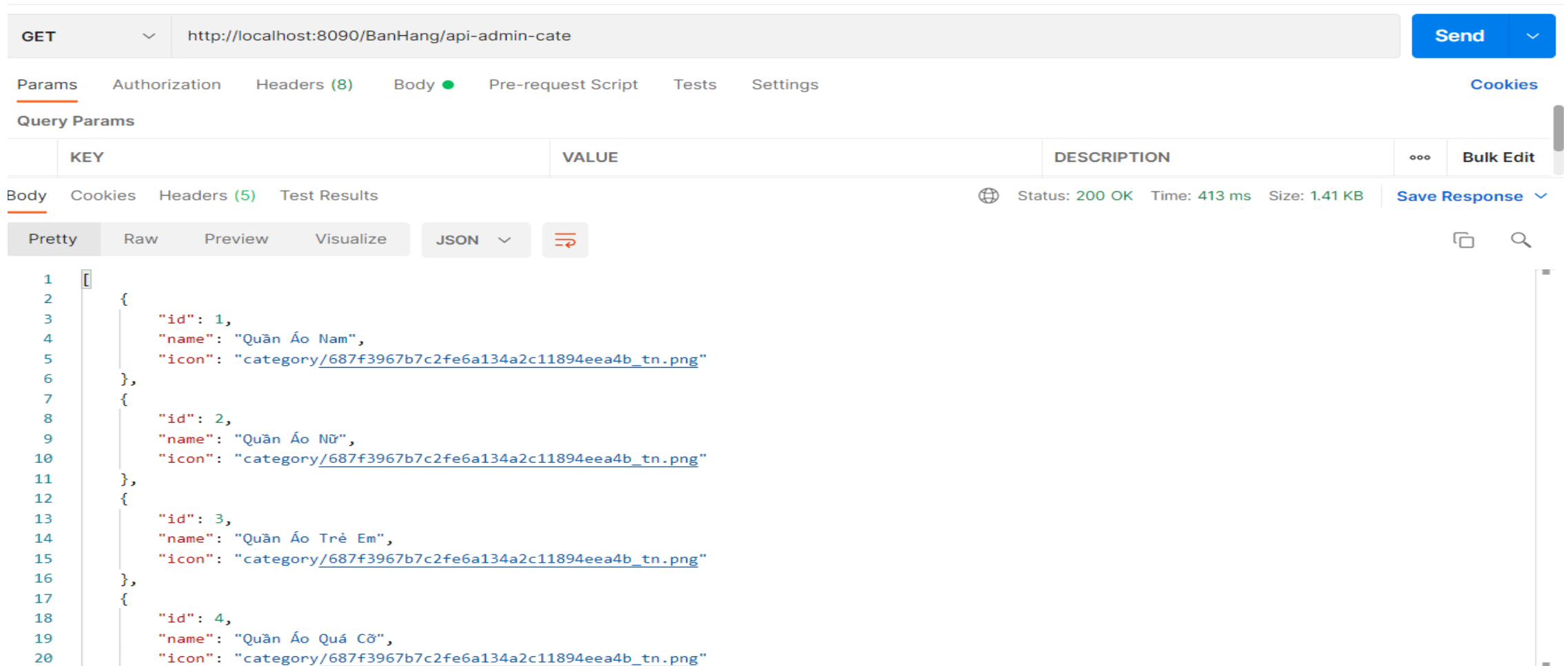
```
    PrintWriter out = resp.getWriter();
```

```
    out.println("Đã xóa thành công");
```

```
    out.close();
```

```
}
```

□ Bước 4: Test kết quả trong Postman



GET ▼ http://localhost:8090/BanHang/api-admin-cate Send ▼

Params Authorization Headers (8) Body ● Pre-request Script Tests Settings Cookies

Query Params

KEY	VALUE	DESCRIPTION	...	Bulk Edit
-----	-------	-------------	-----	-----------

Body Cookies Headers (5) Test Results 🌐 Status: 200 OK Time: 413 ms Size: 1.41 KB Save Response ▼

Pretty Raw Preview Visualize JSON ▼ 🔍

```
1 [
2   {
3     "id": 1,
4     "name": "Quần Áo Nam",
5     "icon": "category/687f3967b7c2fe6a134a2c11894eea4b_tn.png"
6   },
7   {
8     "id": 2,
9     "name": "Quần Áo Nữ",
10    "icon": "category/687f3967b7c2fe6a134a2c11894eea4b_tn.png"
11  },
12  {
13    "id": 3,
14    "name": "Quần Áo Trẻ Em",
15    "icon": "category/687f3967b7c2fe6a134a2c11894eea4b_tn.png"
16  },
17  {
18    "id": 4,
19    "name": "Quần Áo Quá Cỡ",
20    "icon": "category/687f3967b7c2fe6a134a2c11894eea4b_tn.png"
21  }
22 ]
```

LẬP TRÌNH WEB (WEBPR330479)

RESTful API in Spring Boot 3



THS. NGUYỄN HỮU TRUNG

- `@RestController` annotation là sự kết hợp giữa `@Controller` và `@ResponseBody` annotations.
- `@RequestMapping("/api")` để khai báo các url của các api trong controller này sẽ bắt đầu với '/api'
- `@ResponseStatus` để chỉ định controller sẽ trả về status nào đó, ngay cả khi thực hiện thành công
- **`ResponseEntity<T>`**: Đây là cách sử dụng response tốt nhất và mình khuyến khích sử dụng. Nó có ưu điểm hơn hai cách trước như sau:
 - Không cần code dài dòng, class này có sử dụng builder nên dùng khá tiện
 - Với `@ResponseStatus` thì chỉ set cứng status code, tuy nhiên nếu mình muốn status code khác nhau tùy vào điều kiện thì sao. `ResponseEntity<T>` sẽ giải quyết được hết.
 - Có thể vừa đặt data trong body, vừa tùy chỉnh header, vừa,... cùng lúc được luôn.
 - Trong code thì các bạn chỉ cần đổi return type của method từ return gì đó (tạm gọi là X) thành kiểu `ResponseEntity<X>` là được.

Sử dụng @Controller và @ResponseBody Annotation

45

- Với REST API, dữ liệu khi trả về sẽ nằm trong response body, dạng JSON. Nếu dùng @RestController thì không cần @ResponseBody vì nó mặc định có rồi, còn ngược lại @Controller thì phải chỉ định rõ dữ liệu nằm ở đâu trong response.

@Controller

@RequestMapping("v1/api/category")

```
public class CategoryApiController {
```

```
    @Autowired
```

```
    private CategoryRepository repository;
```

```
    @RequestMapping("")
```

```
    @ResponseBody
```

```
    public ResponseEntity<?> list() {
```

```
        return ResponseEntity.ok().body(repository.findAll());
```

```
    }
```

```
}
```

http://localhost:8088/v1/api/category

GET http://localhost:8088/v1/api/category

Params Authorization Headers (7) Body Scripts Settings Cookies

Headers 7 hidden

Key	Value	Description	Bulk Edit	Presets
-----	-------	-------------	-----------	---------

Body Cookies Headers (5) Test Results

200 OK • 125 ms • 740 B • Save Response

Pretty Raw Preview Visualize JSON

```
1 [
2   {
3     "categoryId": 1,
4     "name": "Giày 01a",
5     "products": []
6   },
7   {
8     "categoryId": 2,
9     "name": "Giày 01",
10    "products": []
11  },
12 ]
```

Sử dụng @RestController Annotation

46

- Với REST API, dữ liệu khi trả về sẽ nằm trong response body, dạng JSON. Nếu dùng @RestController thì không cần @ResponseBody vì nó mặc định có rồi.

@RestController

@RequestMapping("v1/api/category")

public class CategoryApiController {

 @Autowired

private CategoryRepository repository;

 @RequestMapping("")

public ResponseEntity<?> list() {

return ResponseEntity.ok().body(repository.findAll());

 }

}

http://localhost:8088/v1/api/category

GET http://localhost:8088/v1/api/category

Params Authorization Headers (7) Body Scripts Settings Cookies

Headers 7 hidden

Key	Value	Description	Bulk Edit	Presets
-----	-------	-------------	-----------	---------

Body Cookies Headers (5) Test Results

200 OK • 125 ms • 740 B • Save Response

Pretty Raw Preview Visualize JSON

```
1 [
2   {
3     "categoryId": 1,
4     "name": "Giày 01a",
5     "products": []
6   },
7   {
8     "categoryId": 2,
9     "name": "Giày 01",
10    "products": []
11  },
12 ]
```

- **ResponseEntity<T>**: Đây là cách sử dụng response tốt nhất và mình khuyến khích sử dụng. Nó có ưu điểm hơn hai cách trước như sau:
 - ▣ Không cần code dài dòng, class này có sử dụng builder nên dùng khá tiện
 - ▣ Với @ResponseStatus thì chỉ set cứng status code, tuy nhiên nếu mình muốn status code khác nhau tùy vào điều kiện thì sao. ResponseEntity<T> sẽ giải quyết được hết.
 - ▣ Có thể vừa đặt data trong body, vừa tùy chỉnh header, vừa,... cùng lúc được luôn.
 - ▣ Trong code thì các bạn chỉ cần đổi return type của method từ return gì đó (tạm gọi là X) thành kiểu ResponseEntity<X> là được.

Sử dụng ResponseEntity<T>

48

```
@GetMapping("/get")
```

```
public ResponseEntity<CategoryEntity>
```

```
getCategory(@RequestParam("name") String categoryname) {
```

```
// Tìm Category trong database bằng name
```

```
Optional<CategoryEntity> category =
```

```
categoryService.findByName(categoryname);
```

```
// Nếu không tìm thấy, trả về message lỗi 404 Not found
```

```
if (category == null)
```

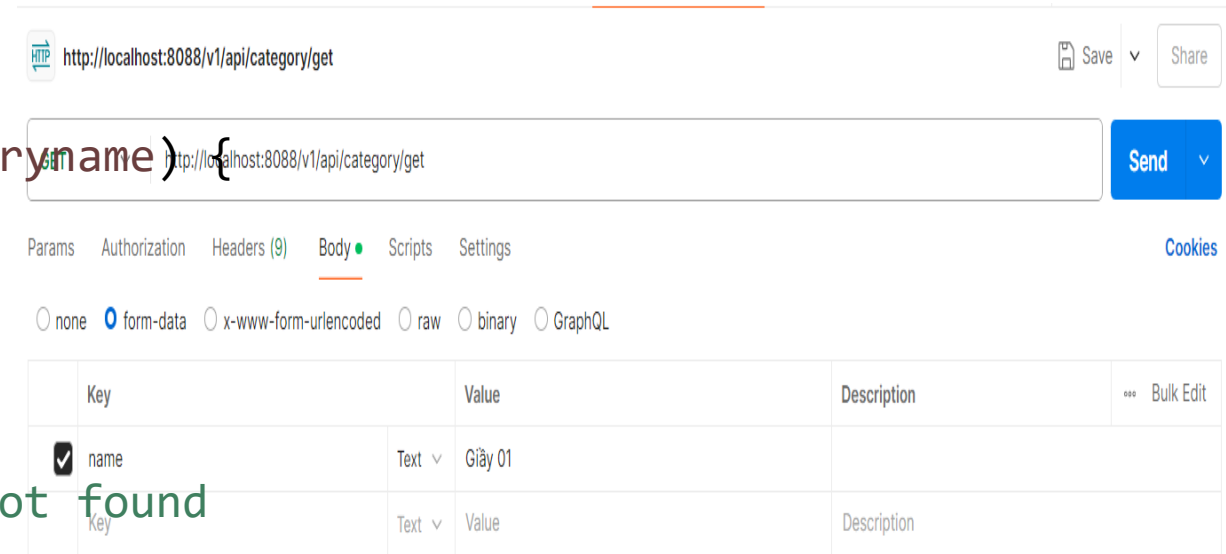
```
// Tạm thời là vậy, thực tế người ta dùng AOP để bắt exception
```

```
return ResponseEntity.notFound().build();
```

```
// Nếu tìm thấy return 200 OK
```

```
return ResponseEntity.ok(category.get());
```

```
}
```

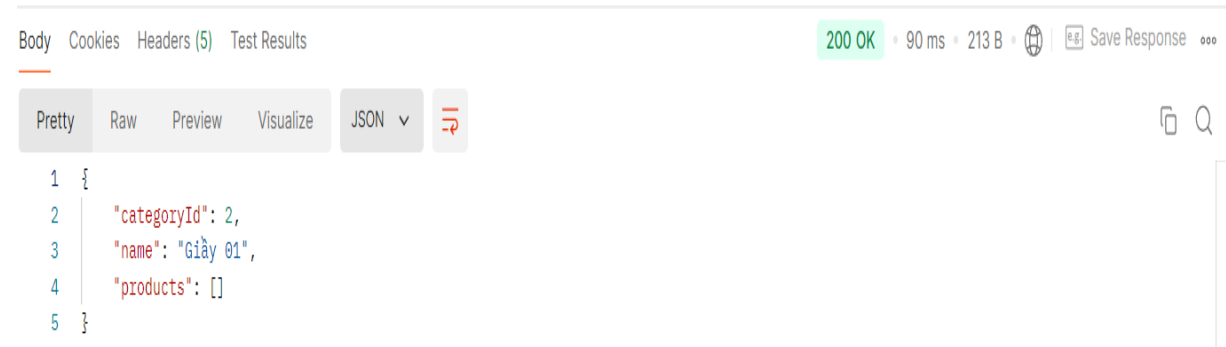


HTTP GET http://localhost:8088/v1/api/category/get

Params Authorization Headers (9) Body Scripts Settings Cookies

☐ none ☒ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

Key	Value	Description	...	Bulk Edit
☒ name	Text Giày 01			
Key	Text Value	Description		



Body Cookies Headers (5) Test Results 200 OK • 90 ms • 213 B • Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "categoryId": 2,
3   "name": "Giày 01",
4   "products": []
5 }
```

□ ResponseEntity<T> có thể dùng 3 cách:

- ▣ Tạo mới object dạng `new ResponseEntity<>(category, HttpStatus.OK)`
- ▣ Dùng dạng static method `ResponseEntity.ok(category)`
- ▣ Dùng dạng builder `ResponseEntity.status(HttpStatus.BAD_REQUEST).body("User not found").build()`

```
@GetMapping("/get")
public ResponseEntity<CategoryEntity> getCategory(@RequestParam("name") String
categoryname) {
    // Tìm Category trong database bằng name
    Optional<CategoryEntity> category = categoryService.findByName(categoryname);
    // Nếu không tìm thấy, trả về message lỗi 404 Not found
    if (category == null)
        // Tạm thời là vậy, thực tế người ta dùng AOP để bắt exception
        // return ResponseEntity.notFound().build();
        return ((HeadersBuilder<BodyBuilder>)
ResponseEntity.status(HttpStatus.BAD_REQUEST).body("User not found")).build();
    // Nếu tìm thấy return 200 OK
    // return ResponseEntity.ok(category.get());
    return new ResponseEntity<CategoryEntity>(category.get(), HttpStatus.OK);
}
```

Sử dụng @RequestBody annotation

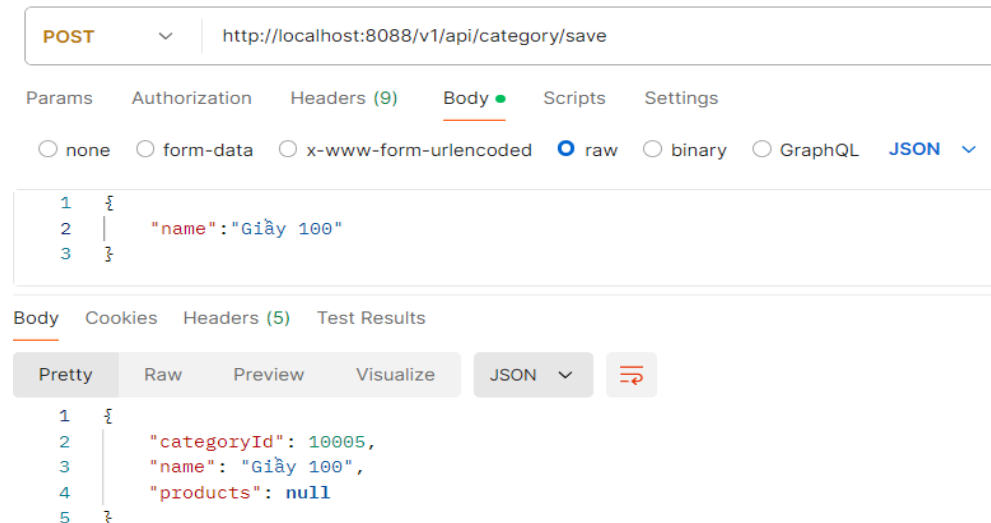
50

- Request method PUT, POST mới có request body, đây là nơi chứa data chính để gửi lên. Thường thì request body sẽ ở dạng JSON hoặc form-data, khi vào controller sẽ được tự động parse ra thành Object (ví dụ Entity, DTO, Models chẳng hạn).

```
@Data
@AllArgsConstructor
@NoArgsConstructor
@Entity
@Table(name = "Categories")
public class CategoryEntity implements Serializable {
    private static final long serialVersionUID = 1L;
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long categoryId;
    @Column(name = "category_name", length = 200, columnDefinition = "nvarchar(200) not null")
    private String name;

    @OneToMany(mappedBy = "category", cascade = CascadeType.ALL )
    private Set<ProductEntity> products;
}
```

```
@PostMapping(value = "/save")
public CategoryEntity saveCate(@Valid
@RequestBody CategoryEntity cate) {
    return categoryService.save(cate);
}
```



Sử dụng @RequestBody annotation

51

□ Update category

```
@RequestMapping(value = "/update/{id}",
method = RequestMethod.PUT)
public ResponseEntity<CategoryEntity>
updateCategory(@PathVariable(value = "id")
Long categoryid,
@Valid @RequestBody CategoryEntity
category) {
Optional<CategoryEntity> cate =
categoryService.findById(categoryid);
if(cate == null) {
return ResponseEntity.notFound().build();
}
cate.get().setName(category.getName());
CategoryEntity updatedContact =
categoryService.save(cate.get());
return ResponseEntity.ok(updatedContact);
}
```

The screenshot shows a REST client interface with the following details:

- Method:** PUT
- URL:** http://localhost:8088/v1/api/category/update/10005
- Body Tab:** Selected, showing a JSON payload:

```
{
  "categoryId": 10005,
  "name": "Giày 100001"
}
```
- Format:** JSON
- Response Tab:** Selected, showing a JSON payload:

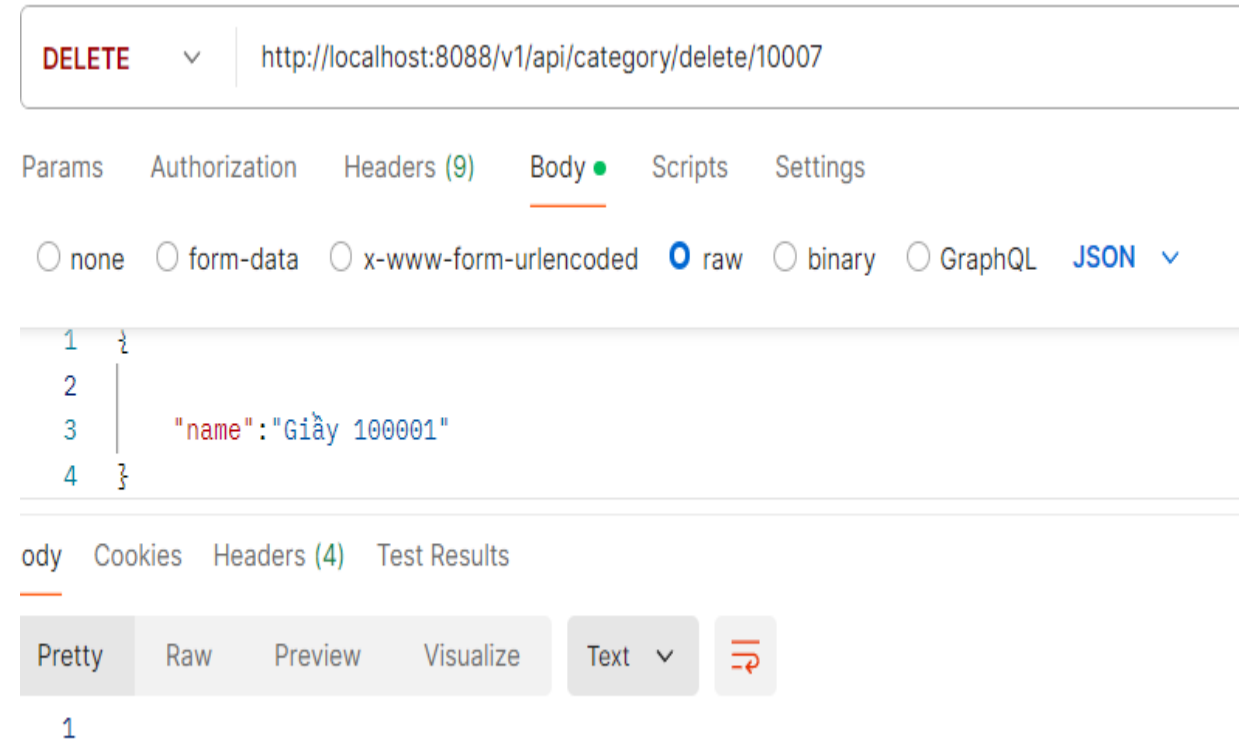
```
{
  "categoryId": 10005,
  "name": "Giày 100001",
  "products": []
}
```

Sử dụng @RequestBody annotation

52

❑ Delete category

```
@DeleteMapping(value = "/contact/{id}")
public ResponseEntity<CategoryEntity>
deleteCategory(@PathVariable(value = "id")
Long categoryid) {
Optional<CategoryEntity> cate =
categoryService.findById(categoryid);
if(cate == null) {
return ResponseEntity.notFound().build();
}
categoryService.delete(cate.get());
return ResponseEntity.ok().build();
}
```



□ Tạo class Response.java để chứa các trường thông tin muốn tùy chỉnh khi response về client.

```
7 @Data
8 @NoArgsConstructor
9 @AllArgsConstructor
10
11 public class Response {
12
13     private Boolean status;
14
15     private String message;
16
17     private Object body;
18
19 }
20
```

```
@GetMapping
public ResponseEntity<> getAllCategory() {
    //return ResponseEntity.ok().body(repository.findAll());
    return new ResponseEntity<Response>(new Response(true, "Thành công", categoryService.findAll()), HttpStatus.OK);
}
```

GET http://localhost:8088/v1/api/category/list

Params Authorization Headers (9) Body ● Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON ▾

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize JSON ▾

```
1 {
2   "status": true,
3   "message": "Thành công",
4   "body": [
5     {
6       "categoryId": 1,
7       "name": "Giày 01a",
8       "products": []
9     },
10    {
11      "categoryId": 2,
12      "name": "Giày 01",
13      "products": []
14    },
15    {
16      "categoryId": 6,
17      "name": "Nguyễn Hữu Trung 1",
18      "products": []
19    },
20  ]
21 }
```

□ Tạo class Response.java để chứa các trường thông tin muốn tùy chỉnh khi response về client.

```
@GetMapping(path = "/get3")
public ResponseEntity<> getCategory(@Validated @RequestParam("id") Long categoryid) {
    Optional<CategoryEntity> category = categoryService.findById(categoryid);

    if (category.isPresent()) {
        //return ResponseEntity.ok().body(category.get());
        return new ResponseEntity<Response>(new Response(true, "Thành công", category.get()), HttpStatus.OK);
    } else {
        //return ResponseEntity.notFound().build();
        return new ResponseEntity<Response>(new Response(false, "Thất bại", null), HttpStatus.NOT_FOUND);
    }
}
```

GET http://localhost:8088/v1/api/category/get3

Params Authorization Headers (9) **Body** Scripts Settings

☐ none ☒ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

	Key	Value
☒	id	Text 1

body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize JSON

```
1 {
2   "status": true,
3   "message": "Thành công",
4   "body": {
5     "categoryId": 1,
6     "name": "Giày 01a",
7     "products": []
8   }
9 }
```

LẬP TRÌNH WEB (WEBPR330479)

Render Rest API to AJAX



THS. NGUYỄN HỮU TRUNG

Gọi thư viện JQuery và viết hàm thực hiện render

56

- ❑ `<script src="https://cdn.jsdelivr.net/npm/jquery@3.7.1/dist/jquery.min.js"></script>`
- ❑ `<script src="/js/main.js"></script>`
- ❑ Hàm xử lý render sẽ viết trong file main.js

```
1=$(document).ready(function() {
2    //Hiển thị thông tin người dùng đăng nhập thành công
3    $.ajax({
4        type: 'GET',
5        url: '/users/me',
6        dataType: 'json',
7        contentType: "application/json; charset=utf-8",
8        beforeSend: function(xhr) {
9            if (localStorage.token) {
10                xhr.setRequestHeader('Authorization', 'Bearer ' + localStorage.token);
11            }
12        },
13        success: function(data) {
14            var json = JSON.stringify(data, null, 4);
15            // $('#profile').html(json);
16            $('#profile').html( data.fullName);
17            $('#images').html(document.getElementById("images").src=data.images);
18            //console.log("SUCCESS : ", data);
19            //alert('Hello ' + data.email + '! You have successfully accessed to /api/profile.');
```

Gọi thư viện JQuery và viết hàm thực hiện render

57

- ❑ `<script src="https://cdn.jsdelivr.net/npm/jquery@3.7.1/dist/jquery.min.js"></script>`
- ❑ `<script src="/js/main.js"></script>`
- ❑ Hàm xử lý render sẽ viết trong file main.js

```
28 //Hàm đăng xuất
29 $('#logout').click(function() {
30     localStorage.clear();
31     window.location.href = "/login";
32 });
33 //hàm Login
34 $('#Login').click(function() {
35     var email = document.getElementById('email').value;
36     var password = document.getElementById('password').value;
37     var basicInfo = JSON.stringify({
38         email:email,
39         password:password |
40     });
41     $.ajax({
42         type: "POST",
43         url: "/auth/login",
44         dataType: 'json',
45         contentType: "application/json; charset=utf-8",
46         data: basicInfo,
47         success: function(data) {
48             localStorage.token = data.token;
49             // alert('Got a token from the server! Token: ' + data.token);
50             window.location.href = "/user/profile";
51         },
52         error: function() {
53             alert("Login Failed");
54         }
55     });
56 });
57 });
```

Gọi thư viện JQuery và viết hàm thực hiện render

58

- `<script src="https://cdn.jsdelivr.net/npm/jquery@3.7.1/dist/jquery.min.js"></script>`
- `<script src="/js/main.js"></script>`
- Gọi ra view

```
<div class="container" style="min-height: 500px">
<form action="" method="post">
<label>Email</label>
<input type="email" name="email" id="email" required="required">
<label>Password</label>
<input type="password" name="password" id="password"
required="required" autocomplete="on">
<button id="Login" type="button">Login</button>
</form>
</div>
```

```
<div class="container" style="min-height: 500px">
<div class="starter-template">
<h1>Spring Boot REST API with AJAX Example</h1>
<img id="images" src="" alt="" width="100">
<div id="profile"></div>
<button id="Logout">Logout</button>
</div>
</div>
```